

ROUND-3 Technical Questions

SECTION 1 – C Programming (Q1–Q20)

Q1. What happens when a C program starts executing?

Answer

Execution starts from the `main()` function.

Before entering `main()`, the operating system performs several initialization tasks:

- Loads the executable into memory.
- Initializes global and static variables.
- Sets up the stack and heap.
- Passes command-line arguments (if any).
- Calls `main()`.

After `main()` finishes, control returns to the operating system.

Follow-up Questions

- Can a C program run without `main()`?
 - In standard C, **No**.
- Which function executes before `main()`?
 - Startup code (CRT - C Runtime) executes before `main()`.

Q2. Explain the complete memory layout of a C program.

Answer

A C program is divided into the following memory segments:

1. **Text Segment**
 - Stores executable instructions.
 - Read-only.
2. **Initialized Data Segment**
 - Stores initialized global and static variables.
3. **BSS Segment**
 - Stores uninitialized global and static variables.
 - Automatically initialized to zero.
4. **Heap**
 - Used for dynamic memory allocation (`malloc()`, `calloc()`).
 - Grows upward.
5. **Stack**
 - Stores local variables, function parameters, and return addresses.

- Grows downward.

Follow-up

Which is faster: Stack or Heap?

Answer: Stack, because memory allocation and deallocation are automatic.

Q3. What is the difference between Compiler, Interpreter, and Assembler?

Answer

Compiler	Interpreter	Assembler
Converts entire source code into machine code	Converts line by line	Converts assembly language into machine code
Faster execution	Slower execution	Fast
Errors shown after compilation	Errors shown line by line	Converts mnemonics into binary

Example:

- GCC → Compiler
- Python → Interpreter
- MASM → Assembler

Q4. Explain the compilation process in C.

Answer

The compilation process consists of four stages:

1. **Preprocessing**
 - Handles #include, #define, macros.
2. **Compilation**
 - Converts C code into assembly code.
3. **Assembly**
 - Converts assembly into object code.
4. **Linking**
 - Links object files and libraries to generate the executable.

Follow-up

Difference between object file and executable file?

- Object file contains machine code but cannot execute independently.
- Executable file is ready to run.

Q5. What is the difference between #include <file.h> and #include "file.h"?

Answer

#include <stdio.h>

- Searches in system directories.

#include "myheader.h"

- Searches in the current directory first, then system directories.

Q6. What are storage classes in C?

Answer

There are four storage classes:

auto

- Default local variable.
- Scope: Local.
- Lifetime: Function.

register

- Suggests storing variable in CPU register.
- Faster access.

static

- Retains value between function calls.
- Lifetime: Entire program.

extern

- Declares a global variable defined in another file.

Follow-up

Can a static local variable retain its value?

Yes.

Q7. Difference between local, global, and static variables.

Answer

Local	Global	Static
Declared inside function	Declared outside function	Can be local or global
Function scope	Entire file	Scope depends on declaration
Created on stack	Stored in data segment	Stored in data segment

Q8. Explain all basic data types in C.

Answer

- char
- int
- float
- double
- void

Typical sizes (may vary by platform):

- char → 1 byte
- int → 4 bytes
- float → 4 bytes
- double → 8 bytes

Follow-up

Difference between float and double?

- double provides higher precision and uses more memory than float.

Q9. What is the difference between signed and unsigned integers?

Answer

Signed int

- Can store both positive and negative values.

Unsigned int

- Stores only non-negative values.
- Uses all bits for magnitude, giving a larger positive range.

Example (32-bit):

- Signed: -2,147,483,648 to 2,147,483,647
- Unsigned: 0 to 4,294,967,295

Q10. Explain operator precedence and associativity.

Answer

Operator precedence determines which operator is evaluated first.

Example:

```
int x = 5 + 3 * 2;
```

Output:

11

because * has higher precedence than +.

Associativity determines evaluation order when operators have the same precedence.

Example:

```
10 - 5 - 2
```

Evaluated as:

```
(10 - 5) - 2
```

because - is left-to-right associative.

Q11. What is the difference between ++i and i++?

Answer

Pre-increment (++i)

- Increments first, then uses the value.

Post-increment (i++)

- Uses the current value first, then increments.

Example:

```
int i = 5;  
printf("%d", ++i);
```

Output:

6

Example:

```
int i = 5;  
printf("%d", i++);
```

Output:

5

After execution, i becomes 6.

Q12. What are lvalue and rvalue?

Answer

lvalue

- Refers to an object with a memory location.
- Can appear on the left side of an assignment.

Example:

```
int x = 10;
```

x is an lvalue.

rvalue

- Temporary value without a persistent memory location.

Example:

```
x = 5 + 3;
```

5 + 3 is an rvalue.

Q13. What is type casting?

Answer

Type casting converts one data type into another.

Example:

```
float x = 5.7;  
int y = (int)x;
```

Output:

5

Types

- Implicit type casting
- Explicit type casting

Q14. Difference between implicit and explicit type conversion.

Answer

Implicit Conversion

- Done automatically by the compiler.

Example:

```
int a = 5;  
float b = a;
```

Explicit Conversion

- Performed by the programmer.

Example:

```
float x = 5.9;  
int y = (int)x;
```

Q15. What are escape sequences?

Answer

Escape sequences represent special characters in strings.

Common examples:

- `\n` → New line

- `\t` → Tab
- `\\` → Backslash
- `\"` → Double quote
- `\0` → Null character

Q16. What is the difference between `scanf()` and `gets()`?

Answer

`scanf()`

- Reads input until whitespace.
- Safer when used correctly.

`gets()`

- Reads an entire line.
- Unsafe because it can cause buffer overflow.
- Removed from the C standard.

Preferred alternative:

`fgets()`

Q17. What is a null pointer?

Answer

A null pointer is a pointer that does not point to any valid memory location.

Example:

```
int *ptr = NULL;
```

Attempting to dereference a null pointer results in undefined behavior.

Q18. What is the difference between `NULL`, `0`, and `'\0'`?

Answer

- `NULL` → Null pointer constant.
- `0` → Integer value zero.
- `'\0'` → Null character used to terminate C strings.

Q19. What is undefined behavior in C?

Answer

Undefined behavior occurs when the C standard does not define what should happen.

Examples:

- Accessing memory out of bounds.
- Dereferencing a null pointer.
- Modifying the same variable multiple times between sequence points.
- Using an uninitialized variable.

The compiler may produce unexpected results.

Q20. What is the difference between sizeof and strlen()?

Answer

sizeof

- Compile-time operator.
- Returns the size in bytes of a data type or object.

Example:

```
char str[] = "Hello";  
printf("%zu", sizeof(str));
```

Output:

6

(includes the null terminator)

strlen()

- Library function from <string.h>.
- Returns the number of characters before the null terminator.

Example:

```
printf("%zu", strlen(str));
```

Output:

5

SECTION 2 – C Programming (Functions, Pointers, Arrays & Memory) (Q21–Q40)

Q21. What is a function in C? Why do we use functions?

Answer

A function is a reusable block of code that performs a specific task.

Advantages:

- Code reusability
- Modularity
- Easy debugging
- Easy maintenance
- Better readability

Example

```
int add(int a,int b)
{
    return a+b;
}
```

Follow-up

Can a function return multiple values?

No, not directly. It can be done using:

- Pointers
- Structures
- Arrays

Q22. Explain Call by Value and Call by Reference.

Answer

Call by Value

A copy of the variable is passed.

Changes do not affect the original variable.

```
void fun(int x)
{
```

```
x=100;  
}
```

Original value remains unchanged.

Call by Reference

The address of the variable is passed.

Changes affect the original variable.

```
void fun(int *x)  
{  
    *x=100;  
}
```

Interview Follow-up

Does C support Call by Reference?

Officially **No**.

C supports only **Call by Value**.

Reference-like behavior is achieved using pointers.

Q23. What is recursion?

Answer

A function calling itself is known as recursion.

Example

```
int fact(int n)  
{  
    if(n==0)  
        return 1;  
  
    return n*fact(n-1);  
}
```

Advantages

- Elegant code
- Tree problems
- Divide & Conquer
- Backtracking

Disadvantages

- Stack overhead
- Slower than iteration
- Stack overflow possible

Q24. What is the difference between recursion and iteration?

Recursion	Iteration
Function calls itself	Uses loops
Uses stack memory	No recursive stack
Slower	Faster
Elegant	Efficient
Can cause Stack Overflow	No Stack Overflow

Q25. What is a pointer?

Answer

A pointer is a variable that stores the address of another variable.

Example

```
int x=10;
```

```
int *p=&x;
```

Here

```
x = 10
```

```
&p = Address
```

```
*p = 10
```

Follow-up

Why are pointers used?

- Dynamic memory allocation
- Arrays
- Linked Lists
- Trees
- Function arguments
- Efficient programming

Q26. Explain pointer declaration.

Example

```
int *p;
```

Meaning

p is a pointer to integer

Example

```
char *p;
```

Pointer to character.

Example

```
float *p;
```

Pointer to float.

Q27. Difference between Address (&) and Dereference (*) operator.

Address Operator (&)

Returns memory address.

Example

```
int a=10;
```

```
printf("%p",&a);
```

Dereference Operator (*)

Accesses value stored at address.

Example

```
int *p=&a;
```

```
printf("%d",*p);
```

Q28. What is NULL Pointer?

Answer

A pointer that points to nothing.

```
int *p=NULL;
```

Advantages

- Prevents accidental access
- Indicates pointer is unused

Follow-up

What happens if we dereference NULL?

Undefined Behavior.

Usually Segmentation Fault.

Q29. What is a Wild Pointer?

Answer

A pointer that has not been initialized.

Example

```
int *p;
```

```
printf("%d",*p);
```

Dangerous because it points to an unknown memory location.

Q30. What is a Dangling Pointer?

Answer

A pointer pointing to deallocated memory.

Example

```
int *p=(int*)malloc(sizeof(int));
```

```
free(p);
```

```
printf("%d",*p);
```

This is undefined behavior.

Prevention

free(p);

p=NULL;

Q31. Difference between NULL Pointer, Wild Pointer and Dangling Pointer.

NULL	Wild	Dangling
Points to nothing	Uninitialized	Points to freed memory
Safe	Dangerous	Dangerous

Q32. What is Pointer Arithmetic?

Answer

Arithmetic operations on pointers.

Allowed

++

--

+

-

Difference

Example

```
int arr[]={10,20,30};
```

```
int *p=arr;
```

```
p++;
```

Now pointer points to 20.

Follow-up

Can we multiply pointers?

No.

Q33. Explain Pointer to Pointer.

Example

```
int x=10;
```

```
int *p=&x;
```

```
int **q=&p;
```

Meaning

q

↓

p

↓

x

Use

- Dynamic 2D arrays
- Triple pointers
- Command line arguments

Q34. Difference between Array and Pointer.

Array	Pointer
Fixed size	Variable
Memory allocated automatically	Can point anywhere
Cannot be reassigned	Can be reassigned
sizeof gives total size	sizeof gives pointer size

Q35. Why is array name considered a constant pointer?

Answer

Array name stores the address of the first element.

```
int arr[5];
```

```
arr == &arr[0]
```

But

`arr++;`

is illegal.

Because array name is constant.

Q36. Difference between

`arr`

and

`&arr`

Answer

`arr`

Address of first element.

Type

`int*`

Whereas

`&arr`

Address of complete array.

Type

`int (*)[size]`

Interviewers ask this very frequently.

Q37. Explain 1D and 2D Arrays in Memory.

1D

10

20

30

40

Continuous memory.

2D

1 2 3

4 5 6

7 8 9

Stored row-wise in C.

Q38. What is Array Decay?

Answer

When an array is passed to a function,

It becomes a pointer.

Example

```
void fun(int arr[])
```

actually means

```
void fun(int *arr)
```

Hence,

```
sizeof(arr)
```

inside function gives pointer size.

Q39. Difference between

```
char str[]="Hello";
```

and

```
char *str="Hello";
```

Answer

First

Stored in array.

Modifiable.

Second

Pointer to string literal.

Modification causes Undefined Behavior.

Example

```
char *s="Hello";
```

```
s[0]='A';
```

Wrong.

Q40. Predict the Output

```
int a=10;
```

```
int *p=&a;
```

```
printf("%d %d",a,*p);
```

Answer

10 10

Follow-up

Output?

```
int a=10;
```

```
int *p=&a;
```

```
*p=20;
```

```
printf("%d",a);
```

Output

20

Reason:

*p directly modifies the memory location of a.

SECTION 3 – Advanced C Programming (Dynamic Memory, Structures, Files, Bit Manipulation) (Q41–Q60)

Q41. What is Dynamic Memory Allocation (DMA)?

Answer

Dynamic Memory Allocation is the process of allocating memory at runtime from the heap.

Functions used:

- malloc()
- calloc()
- realloc()
- free()

Header file:

```
#include <stdlib.h>
```

Advantages

- Memory allocated only when needed.
- Efficient memory utilization.
- Required for dynamic data structures like linked lists, trees, and graphs.

Follow-up

Which memory segment is used?

Answer: Heap.

Q42. Explain malloc().

Answer

malloc() allocates a single block of memory.

Syntax:

```
int *ptr = (int*)malloc(5 * sizeof(int));
```

Characteristics:

- Allocates contiguous memory.
- Memory is **not initialized**.
- Returns NULL if allocation fails.

Follow-up

Why should we check the return value of malloc()?

```
if(ptr == NULL)
{
    printf("Memory Allocation Failed");
}
```

Q43. Difference between malloc() and calloc().

malloc()	calloc()
Allocates one block	Allocates multiple blocks
Memory not initialized	Memory initialized to zero
Faster	Slightly slower
Takes one argument (total bytes)	Takes two arguments (number of elements, size of each element)

Example:

```
int *p = (int*)malloc(5 * sizeof(int));
int *q = (int*)calloc(5, sizeof(int));
```

Q44. What is realloc()?

Answer

realloc() changes the size of previously allocated memory.

Syntax:

```
ptr = realloc(ptr, new_size);
```

Example:

```
int *ptr = (int*)malloc(5*sizeof(int));

ptr = realloc(ptr,10*sizeof(int));
```

Follow-up

What happens if realloc() fails?

It returns NULL, and the original memory block remains valid.

Q45. Why is free() necessary?

Answer

free() releases dynamically allocated memory back to the operating system.

Example

```
free(ptr);  
ptr = NULL;
```

If memory is not freed, it causes a **memory leak**.

Follow-up

What is a memory leak?

Memory allocated but never released.

Q46. What is the difference between Memory Leak and Dangling Pointer?

Memory Leak	Dangling Pointer
Memory not freed	Pointer points to freed memory
Wastes memory	May crash program
No immediate error	Undefined behavior

Q47. What is a Structure?

Answer

A structure is a user-defined data type used to group variables of different data types.

Example

```
struct Student  
{  
    int id;  
    char name[20];  
    float marks;  
};
```

Advantages

- Represents real-world entities.
- Groups related data.
- Easy to manage.

Q48. Difference between Structure and Union.

Structure	Union
Separate memory for each member	Shared memory
Size = Sum of members (plus padding)	Size = Largest member
All members can hold values simultaneously	Only one member is valid at a time

Example

```
struct A
{
    int x;
    float y;
};
union B
{
    int x;
    float y;
};
```

Q49. What is Padding in Structure?

Answer

Padding is extra memory added by the compiler to satisfy alignment requirements.

Example

```
struct Test
{
    char a;
    int b;
};
```

Although:

- char = 1 byte
- int = 4 bytes

The structure size is often **8 bytes**, not 5, because of padding.

Follow-up

Why does the compiler add padding?

To improve memory access speed.

Q50. What is typedef?

Answer

typedef creates an alias for an existing data type.

Example

```
typedef unsigned long ulong;
```

```
ulong x;
```

Structure example

```
typedef struct
{
    int id;
    char name[30];
} Student;
```

Now declare as:

```
Student s;
```

Q51. What is an Enumeration (enum)?

Answer

An enum defines a set of named integer constants.

Example

```
enum Day
{
    MON,
    TUE,
    WED
};
```

Values:

MON = 0
TUE = 1
WED = 2

Q52. What is a Function Pointer?

Answer

A function pointer stores the address of a function.

Example

```
int add(int a,int b)
{
    return a+b;
}
```

```
int (*ptr)(int,int)=add;
```

```
printf("%d",ptr(10,20));
```

Output

30

Applications

- Callback functions
- Interrupt handlers
- Menu-driven programs
- State machines

Q53. What is the difference between Function Pointer and Normal Pointer?

Normal Pointer	Function Pointer
Points to data	Points to function
Stores variable address	Stores function address

Q54. Explain File Handling in C.

Answer

File handling allows reading and writing data to files.

Functions:

- fopen()
- fclose()
- fprintf()
- fscanf()
- fgets()
- fputs()
- fread()
- fwrite()

Q55. What are file opening modes?

Mode	Meaning
r	Read
w	Write
a	Append
r+	Read & Write
w+	Read & Write (overwrite)
a+	Read & Append

Q56. What is Bit Manipulation?

Answer

Bit manipulation means operating directly on binary bits.

Operators:

- & AND
- | OR
- ^ XOR
- ~ NOT
- << Left Shift
- >> Right Shift

Follow-up

Where is bit manipulation used?

- Embedded systems
- Device drivers
- Networking
- Cryptography
- Performance optimization

Q57. How do you check if a number is even or odd using bitwise operators?

```
if(n & 1)
    printf("Odd");
else
    printf("Even");
```

Explanation:

- Last bit = 1 → Odd
- Last bit = 0 → Even

Time Complexity: **O(1)**

Q58. How do you swap two numbers without a temporary variable?

Using XOR:

```
a = a ^ b;
b = a ^ b;
a = a ^ b;
```

Follow-up

What is the limitation?

If a and b refer to the same memory location, this method fails.

Q59. What is the difference between const and volatile?

const

Value cannot be modified.

```
const int x = 10;
```

volatile

Tells the compiler that the variable can change unexpectedly, so it should not optimize accesses.

Example uses:

- Hardware registers

- Interrupt Service Routines (ISR)
- Shared memory

Q60. What are macros? Difference between Macro and Function.

Macro

```
#define SQUARE(x) ((x)*(x))
```

Function

```
int square(int x)
{
    return x*x;
}
```

Macro	Function
Expanded by preprocessor	Executed at runtime
No type checking	Type checking available
Faster (no function call overhead)	Slightly slower
Can cause unexpected side effects	Safer

Interview Follow-up

What is the output?

```
#define SQUARE(x) x*x

printf("%d", SQUARE(2+3));
```

Answer:

11

Expansion:

2 + 3 * 2 + 3

Correct macro:

```
#define SQUARE(x) ((x)*(x))
```

Output:

25

SECTION 4 – C++ & Object-Oriented Programming (Q61–Q80)

Q61. Why was C++ developed when C already existed?

Answer

C++ was developed by **Bjarne Stroustrup** in 1979 as an extension of C.

Limitations of C:

- No Object-Oriented Programming (OOP)
- No Data Hiding
- No Encapsulation
- No Inheritance
- No Polymorphism
- Difficult to manage large software projects

C++ introduced:

- Classes & Objects
- Encapsulation
- Inheritance
- Polymorphism
- Templates
- Exception Handling
- STL

Follow-up

Is C++ completely different from C?

No. C++ is largely backward compatible with C.

Q62. What is Object-Oriented Programming (OOP)?

Answer

OOP is a programming paradigm based on **objects**, which combine data and functions.

The four pillars of OOP are:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

Advantages

- Code Reusability
- Security
- Easy Maintenance
- Modularity
- Scalability

Q63. What is a Class?

Answer

A class is a user-defined blueprint for creating objects.

Example

```
class Student
{
public:
    int id;
    string name;

    void display()
    {
        cout<<id<<" "<<name;
    }
};
```

Follow-up

Does a class occupy memory?

No.

Only objects occupy memory.

Q64. What is an Object?

Answer

An object is an instance of a class.

Example

Student s1;

Here

- Student → Class
- s1 → Object

Follow-up

Where are objects stored?

- Local objects → Stack
- Dynamic objects (new) → Heap
- Global objects → Data Segment

Q65. Difference between Class and Structure in C++

Class	Structure
Default access = private	Default access = public
Used for OOP	Mainly used for data grouping
Supports constructors/destructors	Also supports them in C++, but default access differs

Q66. What is Encapsulation?

Answer

Encapsulation means binding **data and methods** together while restricting direct access using access specifiers.

Example

```
class Bank
{
private:
    int balance;

public:
    void deposit(int amount)
    {
        balance += amount;
    }
};
```

Advantages

- Data Hiding
- Security
- Controlled Access

Q67. Explain Data Abstraction.

Answer

Abstraction means hiding implementation details and exposing only essential functionality.

Example

```
class ATM
{
public:
    void withdraw();
};
```

The user only knows how to call withdraw(), not its internal implementation.

Follow-up

Difference between Encapsulation and Abstraction?

Encapsulation	Abstraction
Hides data	Hides implementation
Achieved using access specifiers	Achieved using abstract classes/interfaces

Q68. What are Access Specifiers?

Answer

Access Specifier	Accessible
public	Everywhere
private	Inside class only
protected	Class + Derived classes

Default access:

- Class → private
- Struct → public

Q69. What is a Constructor?

Answer

A constructor is a special member function automatically called when an object is created.

Characteristics:

- Same name as class
- No return type
- Invoked automatically

Example

```
class Test
{
public:
    Test()
    {
        cout<<"Constructor Called";
    }
};
```

Q70. Types of Constructors

Answer

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor

Example

```
class Student
{
public:
    Student(int x)
    {
    }
};
```

Follow-up

Can constructors be overloaded?

Yes.

Q71. What is a Copy Constructor?

Answer

A copy constructor creates a new object by copying an existing object.

Syntax

```
Student(const Student &obj)
{
}
```

Example

Student s1;

Student s2=s1;

Follow-up

When is the copy constructor called?

- Passing object by value
- Returning object by value
- Initializing one object from another

Q72. What is a Destructor?

Answer

A destructor is automatically called when an object is destroyed.

Characteristics:

- Begins with ~
- No arguments
- No return type
- Only one destructor per class

Example

```
~Student()  
{  
}  
}
```

Purpose

- Release memory
- Close files
- Release resources

Q73. Difference between Constructor and Destructor

Constructor	Destructor
Initializes object	Cleans resources
Called during object creation	Called during object destruction
Multiple constructors allowed	Only one destructor
Can have parameters	Cannot have parameters

Q74. What is the this Pointer?

Answer

this is an implicit pointer available inside every non-static member function.

It points to the current object.

Example

```
class Test
{
    int x;

public:

    void set(int x)
    {
        this->x=x;
    }
};
```

Follow-up

Can this be used in a static function?

No.

Static functions do not belong to any object.

Q75. What is Static Data Member?

Answer

A static data member belongs to the class rather than individual objects.

Example

```
class Test
{
public:
    static int count;
};

int Test::count=0;
```

Only one copy exists regardless of the number of objects.

Q76. What is a Static Member Function?

Answer

A static member function can access only static data members.

Example

```
class Test
{
public:
    static void show()
    {
        cout<<"Hello";
    }
};
```

Call using

Test::show();

Follow-up

Can a static function access non-static variables?

No.

Q77. What is Inheritance?

Answer

Inheritance allows one class to acquire the properties and behaviors of another class.

Example

```
class Animal
{
};

class Dog : public Animal
{
};
```

Advantages

- Code Reusability
- Extensibility
- Maintainability

Q78. Types of Inheritance

Answer

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

Follow-up

Which type may lead to the Diamond Problem?

Multiple and Hybrid Inheritance.

Q79. What is Polymorphism?

Answer

Polymorphism means **one interface, many implementations.**

Types:

- Compile-time Polymorphism
 - Function Overloading
 - Operator Overloading
- Runtime Polymorphism
 - Function Overriding
 - Virtual Functions

Advantages

- Flexibility
- Code Reusability
- Dynamic Behavior

Q80. Difference between Function Overloading and Function Overriding

Function Overloading	Function Overriding
----------------------	---------------------

Same function name, different parameters	Same function signature in base and derived class
Compile-time	Runtime
No inheritance required	Inheritance required
Static binding	Dynamic binding

Example of overloading

```
class Test
{
public:
    void show()
    {
    }

    void show(int x)
    {
    }
};
```

Example of overriding

```
class Base
{
public:
    virtual void show()
    {
    }
};

class Derived : public Base
{
public:
    void show() override
    {
    }
};
```

SECTION 5 – Advanced C++ & OOP (Q81–Q100)

Q81. What is a Virtual Function?

Answer

A virtual function is a member function declared with the virtual keyword in the base class. It enables **runtime polymorphism** (dynamic binding).

```
class Base
{
public:
    virtual void display()
    {
        cout<<"Base";
    }
};

class Derived : public Base
{
public:
    void display()
    {
        cout<<"Derived";
    }
};

int main()
{
    Base *ptr = new Derived();
    ptr->display();
}
```

Output

Derived

Follow-up

Why is virtual required?

Without virtual, the function call is resolved at compile time (static binding).

Q82. What is Late Binding (Dynamic Binding)?

Answer

Late binding means the function call is resolved at **runtime** based on the actual object.

Achieved using:

- Virtual Functions
- Base Class Pointer
- Derived Class Object

Q83. Difference between Compile-time and Runtime Polymorphism.

Compile Time	Runtime
Function Overloading	Function Overriding
Operator Overloading	Virtual Functions
Early Binding	Late Binding
Faster	Slightly slower

Q84. What is a Pure Virtual Function?

Answer

A pure virtual function has **no implementation** in the base class.

Syntax

```
virtual void show() = 0;
```

Example

```
class Shape
{
public:
    virtual void draw()=0;
};
```

Q85. What is an Abstract Class?

Answer

A class containing at least one pure virtual function is called an abstract class.

Example

```
class Animal
{
public:
    virtual void sound()=0;
};
```

Characteristics

- Cannot create objects.
- Used as a base class.
- Supports abstraction.

Q86. Difference between Virtual Function and Pure Virtual Function.

Virtual Function	Pure Virtual Function
Has implementation	No implementation
Object can be created	Object cannot be created
Optional override	Must be overridden

Q87. Why should a Base Class Destructor be Virtual?

Answer

If a derived object is deleted using a base class pointer, the base class destructor should be virtual to ensure both destructors execute.

```
class Base
{
public:
    virtual ~Base()
    {
    }
};
```

Otherwise, resource leaks may occur.

Q88. What is Multiple Inheritance?

Answer

A class inherits from more than one base class.

```
class A
{
};

class B
{
};

class C : public A, public B
{
};
```

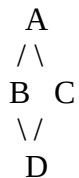
Advantages

- Code reuse
- Combines multiple functionalities

Q89. What is the Diamond Problem?

Answer

It occurs when two classes inherit from the same base class, and another class inherits from both.



Class D gets two copies of class A, causing ambiguity.

Q90. How is the Diamond Problem solved?

Answer

Using **Virtual Inheritance**.

```
class A
{
};

class B : virtual public A
{
};

class C : virtual public A
{
};

class D : public B, public C
{
};
```

Only one copy of class A is created.

Q91. What is a Friend Function?

Answer

A friend function can access private and protected members of a class.

```
class Test
{
private:
    int x=10;

    friend void show(Test);
};

void show(Test t)
{
    cout<<t.x;
}
```

Follow-up

Is a friend function a member function?

No.

Q92. What is Operator Overloading?

Answer

Operator overloading allows existing operators to work with user-defined data types.

Example

```
class Complex
{
public:
    int real;

    Complex operator +(Complex c)
    {
        Complex temp;

        temp.real=real+c.real;

        return temp;
    }
};
```

Advantages

- Cleaner code
- Better readability

Q93. What is Function Template?

Answer

Templates allow writing generic functions.

```
template<typename T>
```

```
T maximum(T a,T b)
{
    return (a>b)?a:b;
}
```

Works for:

- int
- float
- double
- char

without rewriting the function.

Q94. What is Class Template?

Answer

A class template creates generic classes.

```
template<class T>
```

```
class Test
{
public:
```

```
    T data;
};
```

Example

```
Test<int> t1;
```

```
Test<float> t2;
```

Q95. What is Exception Handling?

Answer

Exception handling manages runtime errors gracefully.

Keywords

- try
- throw
- catch

Example

```
try
{
    throw 10;
}
catch(int x)
{
    cout<<x;
}
```

Advantages

- Prevents abrupt program termination
- Improves robustness

Q96. Difference between new/delete and malloc/free.

new/delete	malloc/free
C++	C
Calls constructor/destructor	Does not call constructor/destructor
Type-safe	Not type-safe
Returns correct pointer type	Returns void*

Q97. What are Smart Pointers?

Answer

Smart pointers automatically manage memory and prevent leaks.

Types

- unique_ptr
- shared_ptr
- weak_ptr

Example

```
#include <memory>
```

```
unique_ptr<int> ptr(new int(10));
```

Advantages

- Automatic memory management
- Exception-safe
- Prevents memory leaks

Q98. Difference between `unique_ptr`, `shared_ptr`, and `weak_ptr`.

<code>unique_ptr</code>	<code>shared_ptr</code>	<code>weak_ptr</code>
Single owner	Multiple owners	Observes shared object
Cannot be copied	Reference counting	Doesn't increase reference count
Fastest	Slightly slower	Avoids cyclic references

Q99. What is the Standard Template Library (STL)?

Answer

STL is a library of reusable generic components.

It consists of:

- Containers
- Algorithms
- Iterators
- Function Objects (Functors)
- Allocators

Common containers

- `vector`
- `list`
- `deque`
- `set`
- `map`
- `unordered_map`
- `queue`
- `stack`
- `priority_queue`

Q100. Which STL containers are asked most frequently in interviews?

Answer

vector

- Dynamic array
- Fast random access
- Fast insertion at end

map

- Stores key-value pairs
- Automatically sorted
- Implemented using Red-Black Tree
- Time Complexity: **$O(\log n)$**

unordered_map

- Hash Table
- Average lookup: **$O(1)$**
- Elements not sorted

set

- Stores unique elements
- Sorted automatically
- Time Complexity: **$O(\log n)$**

priority_queue

- Heap-based container
- Default: Max Heap
- Insertion: **$O(\log n)$**
- Deletion: **$O(\log n)$**
- Access top element: **$O(1)$**

Interview Follow-up

When would you use map instead of unordered_map?

Use map when:

- Sorted order is required.
- Range-based operations are needed.
- Predictable iteration order is important.

Use unordered_map when:

- Fast average lookup is the priority.
- Element order does not matter.

SECTION 6 – Data Structures (Q101–Q115)

Q101. What is a Data Structure?

Answer

A Data Structure is a method of organizing and storing data so that it can be accessed and modified efficiently.

Types

Linear Data Structures

- Array
- Linked List
- Stack
- Queue

Non-Linear Data Structures

- Tree
- Graph
- Heap
- Trie

Follow-up

Why do we need Data Structures?

- Efficient searching
- Efficient insertion/deletion
- Better memory utilization
- Faster algorithms

Q102. What is the difference between Array and Linked List?

Answer

Array	Linked List
Contiguous memory	Non-contiguous memory
Fixed size	Dynamic size
Fast random access (O(1))	Sequential access (O(n))
Slow insertion/deletion	Fast insertion/deletion

Less memory overhead	Extra memory for pointers
----------------------	---------------------------

Follow-up

When would you prefer a Linked List over an Array?

When the number of elements changes frequently or when

gave other questions also

Absolutely. In product-based company interviews, they don't just ask "What is a Linked List?" They ask **deep conceptual questions**, scenario-based questions, complexity questions, and implementation questions.

Here are **15 additional high-frequency Data Structure interview questions (Q116–Q130)**.

Q116. Why is Binary Search not possible on a Linked List?

Answer

Binary Search requires **random access** to the middle element in $O(1)$ time.

In a linked list:

- Elements are not stored contiguously.
- Accessing the middle element requires traversing from the head.

Complexity:

- Finding middle: $O(n)$
- Binary Search loses its advantage.

Therefore, Binary Search is inefficient on a Linked List.

Follow-up

Can Binary Search be implemented on a Linked List?

Yes, but overall complexity becomes **$O(n \log n)$** , making linear search preferable.

Q117. What is Time Complexity? How do you calculate it?

Answer

Time Complexity measures how execution time grows with input size (n).

Common complexities:

Complexity	Example
$O(1)$	Array index access
$O(\log n)$	Binary Search
$O(n)$	Linear Search
$O(n \log n)$	Merge Sort
$O(n^2)$	Bubble Sort
$O(2^n)$	Recursive Fibonacci
$O(n!)$	Permutations

Q118. What is Space Complexity?

Answer

Space Complexity is the amount of extra memory an algorithm uses.

Example:

```
int arr[100];
```

Space Complexity = $O(n)$

Recursive functions also consume stack memory.

Q119. What is an Abstract Data Type (ADT)?

Answer

An ADT specifies **what operations are allowed**, not **how they are implemented**.

Examples:

- Stack
- Queue
- Priority Queue

Example:

Stack supports:

- push()
- pop()
- top()

It does not define whether it is implemented using an array or linked list.

Q120. What is the difference between Stack and Queue?

Stack	Queue
LIFO	FIFO
push(), pop()	enqueue(), dequeue()
One end used	Two ends used
Top pointer	Front & Rear

Applications:

Stack:

- Function calls
- Undo operation
- Expression evaluation

Queue:

- CPU Scheduling
- Printer Queue
- BFS
- Networking

Q121. Why is Stack called LIFO?

Answer

Last element inserted is removed first.

Example:

Push 10
Push 20
Push 30

Pop → 30

Applications:

- Recursion
- Browser History
- Undo/Redo

Q122. Explain Queue Operations.

Answer

Operations:

- Enqueue
- Dequeue
- Front
- Rear
- isEmpty()
- isFull()

Complexity:

Operation Complexity

Enqueue O(1)

Dequeue O(1)

Front O(1)

Q123. Difference between Linear Queue and Circular Queue.

Answer

Linear Queue

- Wastes space after deletions.
- Rear reaches end quickly.

Circular Queue

- Reuses empty spaces.
- Better memory utilization.

Applications:

- CPU Scheduling
- Buffers
- Embedded Systems

Q124. What is a Deque?

Answer

Deque (Double Ended Queue)

Insertion and deletion can happen at both ends.

Operations:

- push_front()
- push_back()
- pop_front()
- pop_back()

Applications:

- Sliding Window
- LRU Cache

Q125. What is a Priority Queue?

Answer

Elements are removed based on priority rather than insertion order.

Usually implemented using a Heap.

Complexity:

Insertion $\rightarrow O(\log n)$

Deletion $\rightarrow O(\log n)$

Top $\rightarrow O(1)$

Applications:

- Dijkstra
- CPU Scheduling

- Huffman Coding

Q126. Why is a Binary Tree called Binary?

Answer

Each node has at most two children.

```
  10
 /  \
5    20
```

Children:

- Left Child
- Right Child

Q127. Difference between Binary Tree and Binary Search Tree.

Binary Tree

BST

No ordering

Left < Root < Right

Searching $O(n)$

Searching $O(\log n)$ (average)

Used for hierarchy

Used for searching

Q128. What is Tree Traversal?

Answer

Traversal means visiting every node exactly once.

Types:

Preorder

Root Left Right

Inorder

Left Root Right

Postorder

Left Right Root

Level Order

Breadth First Search

Q129. Why is Inorder Traversal important in BST?

Answer

Inorder Traversal of a BST always produces elements in **sorted order**.

Example

```
  10
 /  \
5    15
```

Inorder:

5 10 15

This property is frequently used in searching and sorting applications.

Q130. What is a Complete Binary Tree?

Answer

A Complete Binary Tree is a binary tree in which:

- Every level is completely filled except possibly the last.
- Nodes in the last level are filled from left to right.

Example:

```
  1
 /  \
2    3
/\  /\
4 5 6
```

Applications:

- Heap
- Priority Queue

These are exactly the types of conceptual Data Structure questions that interviewers ask after basic definitions.

The remaining **Data Structures section (Q131–Q150)** can cover advanced topics such as:

- AVL Trees
- Red-Black Trees
- Heap
- Hash Table
- Collision Handling
- Trie
- Graph Representations
- BFS vs DFS
- Disjoint Set Union (Union-Find)
- Segment Tree
- Fenwick Tree
- LRU Cache
- Complexity-based scenario questions
- Real-world applications of each data structure

These advanced questions are commonly asked in product-based technical interviews to evaluate deeper understanding beyond textbook definitions.

SECTION 6 – Data Structures (Q101–Q115) completed this section you give only 2 questions

You're right. I only completed **Q101 and Q102**. Below are the **remaining questions (Q103–Q115)** to properly complete **Section 6 – Data Structures**.

Q103. What is a Linked List?

Answer

A Linked List is a linear data structure where each node contains:

- Data
- Pointer to the next node

Unlike arrays, linked list elements are stored in non-contiguous memory locations.

Advantages:

- Dynamic size
- Efficient insertion and deletion
- Better memory utilization

Disadvantages:

- No random access
- Extra memory required for pointers

Q104. What are the types of Linked Lists?

Answer

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List
4. Circular Doubly Linked List

Follow-up

Which linked list allows traversal in both directions?

Answer: Doubly Linked List.

Q105. Difference between Singly and Doubly Linked List.

Singly Linked List	Doubly Linked List
One pointer (next)	Two pointers (prev, next)
Less memory	More memory
Forward traversal	Forward & Backward traversal
Simpler implementation	More complex

Q106. What is the time complexity of Linked List operations?

Operation	Time Complexity
Access	O(n)
Search	O(n)
Insert at Beginning	O(1)
Insert at End	O(n) (O(1) with tail pointer)
Delete at Beginning	O(1)
Delete at End	O(n)

Q107. What is a Stack?

Answer

A Stack is a linear data structure that follows the **LIFO (Last In, First Out)** principle.

Operations:

- Push
- Pop
- Peek/Top
- isEmpty()

Applications:

- Function Calls
- Undo/Redo
- Expression Evaluation
- Backtracking

Q108. What is a Queue?

Answer

A Queue is a linear data structure that follows the **FIFO (First In, First Out)** principle.

Operations:

- Enqueue
- Dequeue
- Front
- Rear

Applications:

- CPU Scheduling
- Printer Queue
- BFS
- Networking

Q109. Difference between Stack and Queue.

Stack	Queue
LIFO	FIFO
Push & Pop	Enqueue & Dequeue
One end	Two ends
Top Pointer	Front & Rear

Q110. What is a Tree?

Answer

A Tree is a hierarchical non-linear data structure consisting of nodes connected by edges.

Terminology:

- Root Node
- Parent Node
- Child Node
- Leaf Node
- Height
- Depth

Applications:

- File System
- XML Parsing
- Database Indexing

Q111. What is a Binary Tree?

Answer

A Binary Tree is a tree in which each node has at most two children:

- Left Child
- Right Child

Example:

```
  10
 /  \
5    20
```

Types:

- Full Binary Tree
- Complete Binary Tree
- Perfect Binary Tree
- Balanced Binary Tree

Q112. What is a Binary Search Tree (BST)?

Answer

A BST is a binary tree where:

- Left subtree contains smaller values.
- Right subtree contains larger values.

Property:

Left < Root < Right

Searching:

- Average Case $\rightarrow O(\log n)$
- Worst Case $\rightarrow O(n)$

Applications:

- Searching
- Sorting
- Symbol Tables

Q113. What is a Heap?

Answer

A Heap is a complete binary tree that satisfies the Heap Property.

Types:

- Max Heap $\rightarrow \text{Parent} \geq \text{Children}$
- Min Heap $\rightarrow \text{Parent} \leq \text{Children}$

Applications:

- Priority Queue
- Heap Sort
- Dijkstra's Algorithm

Time Complexity:

- Insert $\rightarrow O(\log n)$
- Delete $\rightarrow O(\log n)$
- Peek $\rightarrow O(1)$

Q114. What is Hashing?

Answer

Hashing maps a key to an index using a hash function.

Advantages:

- Very fast searching
- Fast insertion
- Fast deletion

Average Complexity:

- Search $\rightarrow O(1)$
- Insert $\rightarrow O(1)$
- Delete $\rightarrow O(1)$

Applications:

- Dictionaries
- Caches
- Database Indexing
- Compiler Symbol Tables

Q115. What is Collision in Hashing? How is it resolved?

Answer

A collision occurs when two different keys produce the same hash index.

Example:

Hash(15) = 5

Hash(25) = 5

Collision Resolution Techniques:

1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Double Hashing

Follow-up

Which collision resolution technique is most commonly used?

Answer: Separate Chaining, because it is simple to implement and performs well even with a higher load factor.